

Lecture 19: Buffer Management and Storage Control

Amit Kumar Dhar

October 24, 2025

Storage Control & The Buffer Pool

Memory-Mapped I/O

Buffer Replacement Policies

Interaction with the OS

Storage Control & The Buffer Pool

The Need for Storage Control

A DBMS needs to manage storage for two primary reasons:

- **Spatial Control:** Where to place data on disk.
 - Goal: Place related data close together to minimize disk arm movement (seek time).
 - Example: Storing all records of a table in contiguous blocks.
- **Temporal Control:** When to move data between disk and memory.
 - Goal: Keep frequently accessed data in memory to avoid slow disk I/O.
 - This is the primary job of the **Buffer Manager**.

Effective storage control is the most critical factor for DBMS performance.

The Buffer Pool's Role

The buffer pool is a region of main memory used to cache disk pages. It acts as an intermediary between the CPU/Query Executor and the slow disk subsystem.

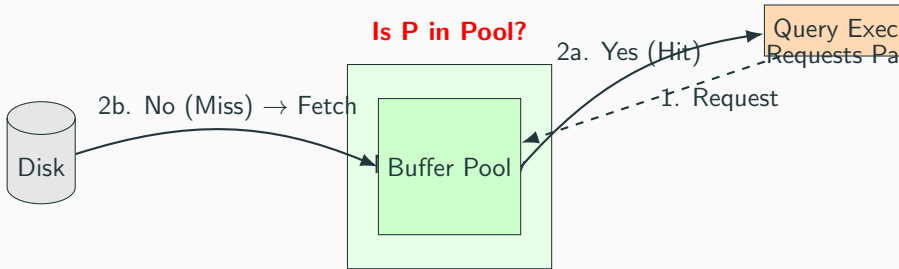
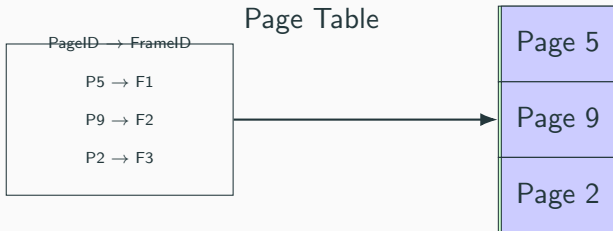


Figure 1: The buffer pool intercepts page requests, avoiding disk I/O on a cache hit.

Buffer Pool Organization

The buffer pool consists of three main components:

- **Frames:** An array of fixed-size slots in memory, each of which can hold one disk page.
- **Page Table:** An in-memory hash table that maps a Page ID (from disk) to the Frame ID where it is currently stored.
- **Metadata:** Information about each page, often stored in the page table (or a parallel array).



Page Table Metadata & Latches

For each page in the buffer pool, the page table (or a control block) stores crucial metadata:

- **Dirty Bit:** Is the in-memory copy newer than the on-disk copy? If true, it must be written to disk before the frame can be reused.
- **Pin Count (or Reference Count):** How many concurrent operations are currently using this page? A page with a pin count ≥ 0 cannot be evicted.
- **Access History:** Information for the replacement policy (e.g., last access time for LRU).

Latches (or Mutexes):

- The buffer pool is a shared data structure. Multiple threads might try to access it at once.
- A **latch** is a lightweight, short-duration lock that protects the

Locks vs. Latches

Latches	Locks
Protects internal memory structures	Protects database content (tuple)
Held for very short durations	Held for transaction duration
Managed by DBMS internals	Managed by the Lock Manager
No deadlock detection needed	Requires deadlock detection/prevention
Fast, often implemented as spinlocks	Slower, involves more complex logic

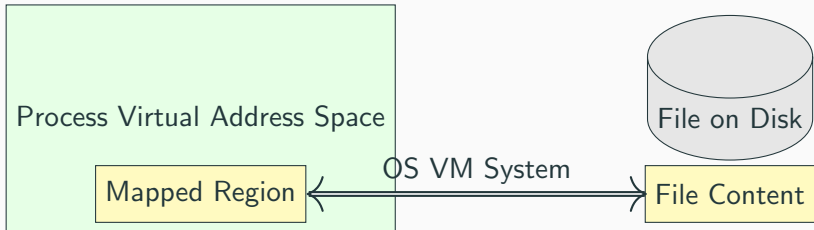
Table 1: Key differences between concurrency control mechanisms.

Example: A thread acquires a **latch** on the page table to find a page's frame. Once found, it releases the latch and acquires a **lock** on a record within that page.

Memory-Mapped I/O

Memory-Mapped I/O (mmap)

- An alternative to the DBMS buffer pool is to use the OS's virtual memory system via the 'mmap()' system call.
- 'mmap()' maps a file on disk directly into the process's virtual address space.
- Accessing a memory address in the mapped region triggers a page fault, and the OS automatically loads the corresponding page from the file into memory.



The OS provides system calls to control memory-mapped regions:

- `madvise()`: Give performance hints to the OS about how you expect to use a memory region (e.g., 'MADV_SEQUENTIAL' for sequential scans, 'MADV_RANDOM' for random access).
- `mlock()`: Pin a memory region, preventing the OS from swapping it out to disk. Useful for keeping critical data structures in memory.
- `msync()`: Flushes changes in the memory region back to the underlying file on disk, similar to 'fsync()'. This is crucial for durability.

Problems with mmap for a DBMS

While simple, 'mmap' presents serious challenges for a robust DBMS:

- **Transaction Safety:** It's hard to implement Write-Ahead Logging (WAL). A process might crash after the OS writes a dirty page to disk but before the corresponding log record is written.
- **I/O Stalls:** If a thread accesses an un-cached page, it will stall on a page fault. A DBMS buffer manager can use non-blocking I/O to perform other work instead of stalling.
- **Performance Issues:** The OS's page replacement policy is generic (like LRU) and unaware of database access patterns, leading to poor caching decisions (e.g., sequential flooding).
- **Error Handling:** Limited ability to handle I/O errors gracefully compared to direct file I/O.

Buffer Replacement Policies

Buffer Replacement Policies

When the buffer is full, which page do we evict?

- **Goal:** Evict the page that is least likely to be needed in the near future.
- Important Things:
 - Correctness
 - Accuracy
 - Speed
 - Meta-data overhead
- We'll look at a few key policies:
 1. LRU (Least Recently Used)
 2. Clock
 3. LRU-K
 4. ARC (Adaptive Replacement Cache)

LRU (Least Recently Used)

- Evict the page that has not been accessed for the longest period of time.
- Maintained by keeping a timestamp of the last access for each page or using a linked list.

Example: Buffer size = 3 frames. Request sequence: A, B, C, D, B, E

1. Req A: [A]
2. Req B: [A, B]
3. Req C: [A, B, C] (A is LRU)
4. Req D: [D, B, C] (Evict A)
5. Req B: [D, C, B] (B becomes MRU)
6. Req E: [D, E, B] (Evict C)

Clock Algorithm

- A more efficient approximation of LRU.
- Frames are arranged in a circle with a "clock hand".
- Each frame has a reference bit (use bit).
- When a page is accessed, its reference bit is set to 1.
- To find a victim, the clock hand sweeps over the frames:
 - If reference bit is 1, set it to 0 and advance the hand.
 - If reference bit is 0, this is the victim. Evict the page.

This gives a page a "second chance" before it's evicted.

Problem: Sequential Flooding

Both LRU and Clock perform poorly for large sequential scans.

- A scan of a large table can fill the entire buffer pool with pages that will only be used once.
- This "flooding" evicts all the "hot" (frequently used) pages from the cache.

Example: Buffer size = 100. Scan a 1000-page table.

- The first 100 pages of the scan fill the buffer, evicting useful pages P1, P2, ...
- The next 100 pages of the scan evict the first 100 scan pages.
- By the end, the buffer contains the LAST 100 pages of the table, which are useless, and all previously cached hot pages are gone.

LRU-K: Improving on LRU

- LRU-K addresses the sequential flooding problem by considering more than just the last access time.
- It keeps track of the timestamps of the **last K accesses** for each page.
- The "distance" to a page is the time since its K-th most recent access.
- The victim is the page with the largest backward K-distance.
- A page accessed only once (like in a scan) will have an infinite K-distance (for $K > 1$) and will be a prime candidate for eviction.

LRU-K: Improving on LRU

Example (K=2):

- Page A accessed at $t=1, 5, 10$. Backward-2 distance is from $t=5$.
- Page B accessed at $t=9$. Backward-2 distance is infinite.
- Page B is a better candidate for eviction than A.

Other Policies & Considerations

- **ARC (Adaptive Replacement Cache):** Maintains two LRU lists: one for recently accessed pages (T1) and one for frequently accessed pages (T2). It adaptively adjusts the size of these lists based on the workload, making it robust to both scans and random access.
- **Localization (per-process buffers):** Assign a portion of the buffer pool to a specific query to ensure it doesn't interfere with others.
- **Priority Hints:** The query optimizer can "hint" to the buffer manager that certain pages (e.g., index root pages) are more important and should not be evicted.

Interaction with the OS

Taking Care of Dirty Pages

- A page is "dirty" if it has been modified in the buffer pool but not yet written back to disk.
- The buffer manager cannot evict a dirty page. It must first be **flushed** to disk.
- To ensure transaction durability (the 'D' in ACID), the DBMS uses **Write-Ahead Logging (WAL)**.
 1. The log record describing the change is written to persistent storage (disk).
 2. Only then can the dirty data page be flushed to disk.
- The DBMS often has background writer processes that periodically flush dirty pages to disk to create clean frames for future use.

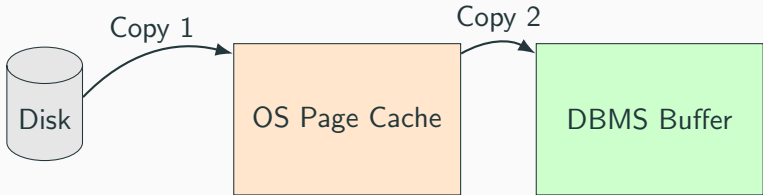
DBMS vs. OS Disk Scheduling

Why not let the OS handle I/O scheduling?

- The OS has limited information. It sees a stream of read/write requests from different processes.
- The DBMS has high-level knowledge of the query plan and data layout.
 - It can predict future page requests for a query.
 - It can group I/O for different queries to optimize disk arm movement.
 - It can prioritize I/O for interactive queries over background tasks.
- This allows the DBMS to make much more intelligent I/O scheduling decisions than a general-purpose OS.

Avoiding the OS Page Cache

- The OS also maintains its own file system cache (the "page cache").
- If the DBMS issues standard 'read()' calls, data would be copied from disk → OS page cache → DBMS buffer pool.
- This "double buffering" wastes memory and CPU cycles.



- **Solution:** DBMSs open database files using a special flag like `O_DIRECT` on Linux.

Prefetching and Scan Sharing

Two techniques that leverage the DBMS's high-level knowledge:

- **Prefetching:** The DBMS anticipates which pages a query will need soon and issues asynchronous I/O requests to load them into the buffer pool before they are explicitly requested.
 - **Example:** For `SELECT * FROM Employees;`, while the query processor is working on page 1, the buffer manager can prefetch pages 2, 3, 4, etc.
- **Scan Sharing:** If multiple queries are scanning the same table, the DBMS can use a single disk scan to service all of them.
 - **Example:**
 - Q1: `SELECT AVG(salary) FROM Employees WHERE age > 40;`
 - Q2: `SELECT COUNT(*) FROM Employees WHERE dept = 'Sales';`
 - The DBMS can perform one full scan of the Employees table and feed the relevant tuples to both query operators

Questions?